

PRACTICAL 10

Q1. Inode, Hard Links and Symbolic Links

Aim

To investigate inode structures using `ls -i` and `stat`, and create hard and symbolic links.

Theory

An inode stores information about a file such as its permissions, size, owner, timestamps and link count.

Commands

Create a file:

```
amrutha@blue:~$ echo "Hello Linux" > filedemo.txt
amrutha@blue:~$ ls -i filedemo.txt
24313 filedemo.txt
amrutha@blue:~$ stat filedemo.txt
  File: filedemo.txt
  size: 12          Blocks: 8          IO Block: 4096   regular file
Device: 8,48      Inode: 24313         Links: 1
Access: (0644/-rw-r--r--)  Uid: ( 1000/ amrutha)   Gid: ( 1000/ amrutha)
Access: 2026-09-27 19:15:24.654451020 +0000
Modify: 2026-09-27 19:15:24.654451020 +0000
Change: 2026-09-27 19:15:24.654451020 +0000
 Birth: 2026-09-27 19:15:24.654451020 +0000
```

`echo "Hello Linux" > file1.txt`

Check inode:

`ls -i file1.txt`

View complete information:

`stat file1.txt`

Create Hard Link

```
amrutha@blue:~$ ln filedemo.txt hardlink2.txt
amrutha@blue:~$ ls -li filedemo.txt hardlink2.txt
24313 -rw-r--r-- 2 amrutha amrutha 12 Sep 27 19:15 filedemo.txt
24313 -rw-r--r-- 2 amrutha amrutha 12 Sep 27 19:15 hardlink2.txt
```

`ln file1.txt hardlink.txt`

Check:

`ls -li file1.txt hardlink.txt`

Example Output

123456 file1.txt

123456 hardlink.txt

Both have the **same inode number**.

Create Symbolic Link

```
amrutha@blue:~$ ln -s filedemo.txt symlink.txt
amrutha@blue:~$ ls -li filedemo.txt hardlink2.txt symlink.txt
24313 -rw-r--r-- 2 amrutha amrutha 12 Sep 27 19:15 filedemo.txt
24313 -rw-r--r-- 2 amrutha amrutha 12 Sep 27 19:15 hardlink2.txt
25250 lrwxrwxrwx 1 amrutha amrutha 12 Sep 27 19:18 symlink.txt -> filedemo.t
xt
```

`ln -s file1.txt symlink.txt`

Check:

`ls -li file1.txt hardlink.txt symlink.txt`

The symbolic link has a **different inode number**.

Comparison

Hard Link	Symbolic Link
Same inode as original	Different inode
Points directly to inode	Points to file path
<code>ln file1 hardlink</code>	<code>ln -s file1 symlink</code>
Works even if original name is removed	Can become broken

Observation

Hard links share the same inode, while a symbolic link has its own inode.

Result

Thus, inode information was examined and hard and symbolic links were successfully created.

Q2. File I/O Using mmap()

Aim

To perform file reading using `mmap()` and compare it with traditional `read()` and `write()` operations.

Theory

`mmap()` maps a file into the process's memory address space. The file can then be accessed through the memory address.

Program

```
#include <stdio.h>
#include <fcntl.h>
#include <unistd.h>
#include <sys/mman.h>
#include <sys/stat.h>

int main() {
    int fd;
    struct stat st;
    char *data;

    fd = open("input.txt", O_RDONLY);

    if (fd < 0) {
        perror("Error opening file");
        return 1;
    }

    fstat(fd, &st);

    data = mmap(NULL, st.st_size, PROT_READ,
                MAP_PRIVATE, fd, 0);

    if (data == MAP_FAILED) {
        perror("mmap failed");
        close(fd);
        return 1;
    }

    printf("File contents:\n");
    write(STDOUT_FILENO, data, st.st_size);

    munmap(data, st.st_size);
    close(fd);

    return 0;
}
```

Execution

echo "Hello from mmap!" > input.txt

gcc mmap.c -o mmap

./mmap

Output

File contents:

Hello from mmap!

Comparison

read()/write()	mmap()
Uses system calls	Maps file into memory
Data is explicitly read	Data is accessed through memory
Simple normal I/O	Useful for memory-based/random access

Observation

The file was mapped into memory using mmap() and its contents were accessed.

Result

Thus, file reading using memory-mapped I/O was successfully implemented.